

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

etsinf

Escola Tècnica Superior d'Enginyeria Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

De señal acústica a partitura: transcripción automática de piano mediante aprendizaje profundo

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Autor: Jorge Rodríguez González

Tutor: Alfons Juan Císcar

Curso 2025-2026

Resum

La transcripció automàtica de música (AMT) constitueix un dels problemes centrals de la recuperació d'informació musical (MIR), l'objectiu de la qual és convertir un senyal d'àudio en una representació simbòlica de la música interpretada. En el cas del piano, l'objectiu és obtindre a partir de l'àudio una representació MIDI que capture amb precisió els esdeveniments de nota –to, inici, final i dinàmica– presents en la interpretació. Aquest treball aborda la replicació i l'avaluació de dos sistemes de l'estat de l'art per a la transcripció automàtica. En primer lloc, s'entrena un model MFT-FiLM (*Multitask Frequency-Time Feature-wise Linear Modulation*) que combina convolucions espectrals i xarxes recurrents bidireccionals per a l'estimació conjunta d'*onsets*, *offsets*, *frames* i velocitats. A continuació, s'entrena un model Transkun, un sistema basat en *transformers* que aborda la transcripció com un problema de traducció de seqüències. Tots dos models s'avaluen de forma quantitativa sobre el conjunt de test de MAESTRO seguint el protocol estàndard (`mir_eval`). Addicionalment, s'avalua la capacitat de generalització de tots dos models sobre el *dataset* PiJAMA, la distribució de repertori i àudio del qual difereix de MAESTRO, i s'exploren modificacions experimentals orientades a la millora del rendiment dels models.

Paraules clau: Transcripció automàtica, xarxes convolucionals, *transformers*, *onset*, *offset*, *frame*, modulació lineal, MIDI, espectrograma, MAESTRO, PiJAMA, piano

Resumen

La transcripción automática de música (AMT) constituye uno de los problemas centrales de la recuperación de información musical (MIR), cuyo objetivo es convertir una señal de audio en una representación simbólica de la música interpretada. En el caso del piano, el objetivo es obtener a partir del audio una representación MIDI que capture con precisión los eventos de nota –tono, inicio, final y dinámica– presentes en la interpretación. Este trabajo aborda la replicación y evaluación de dos sistemas del estado del arte para la transcripción automática. En primer lugar, se entrena un modelo MFT-FiLM (*Multitask Frequency-Time Feature-wise Linear Modulation*) que combina convoluciones espectrales y redes recurrentes bidireccionales para la estimación conjunta de *onsets*, *offsets*, *frames* y velocidades. A continuación, se entrena un modelo Transkun, un sistema basado en transformers que aborda la transcripción como un problema de traducción de secuencias. Ambos modelos se evalúan de forma cuantitativa sobre el conjunto de test de MAESTRO siguiendo el protocolo estándar (`mir_eval`). Adicionalmente se evalúa la capacidad de generalización de ambos modelos sobre el dataset PiJAMA, cuya distribución de repertorio y audio difiere de MAESTRO, y se exploran modificaciones experimentales orientadas a la mejora del rendimiento de los modelos.

Palabras clave: Transcripción automática, redes convolucionales, *transformers*, *onset*, *offset*, *frame*, modulación lineal, MIDI, espectrograma, MAESTRO, PiJAMA, piano

Abstract

Automatic Music Transcription (AMT) constitutes one of the core problems in Music Information Retrieval (MIR), aiming to convert an audio signal into a symbolic representation of the performed music. In the case of the piano, the goal is to obtain a MIDI representation from the audio that accurately captures the note events –pitch, onset, offset, and dynamics– present in the performance. This work addresses the replication and evaluation of two state-of-the-art systems for automatic transcription. First, an MFT-FiLM

(Multitask Frequency-Time Feature-wise Linear Modulation) model is trained, which combines spectral convolutions and bidirectional recurrent networks for the joint estimation of onsets, offsets, frames, and velocities. Next, a Transkun model is trained, a *transformer*-based system that approaches transcription as a sequence translation problem. Both models are quantitatively evaluated on the MAESTRO test set following the standard protocol (`mir_eval`). Additionally, the generalization capability of both models is evaluated on the PiJAMA dataset, whose repertoire and audio distribution differs from MAESTRO, and experimental modifications aimed at improving the models' performance are explored.

Key words: Automatic transcription, convolutional networks, *transformers*, *onset*, *offset*, *frame*, linear modulation, MIDI, spectrogram, MAESTRO, PiJAMA, piano

Índice general

Índice general v
Índice de figuras vii
Índice de tablas vii

1 Introducción 1
1.1 Motivación . . . 1
1.2 Objetivos . . . 1
1.3 Estructura de la memoria . . . 1
2 Preliminares 3
2.1 ?? ???? ???? ? ?? ?? . . . 3
3 MAESTRO 5
3.1 Introducción y contexto . . . 5
3.2 Origen: International Piano-e-Competition . . . 5
3.2.1 Yamaha Disklavier . . . 5
3.3 Adquisición y anotación . . . 6
3.3.1 Captura del audio . . . 6
3.3.2 MIDI y anotación . . . 6
3.4 Características del dataset . . . 7
3.4.1 Composición . . . 7
3.4.2 Particiones . . . 7
3.5 Evolución del dataset . . . 8
3.6 Distribución y licencia . . . 8
3.7 Limitaciones . . . 9
3.8 Relevancia para este trabajo . . . 9
3.9 Evaluación estándar . . . 10
3.9.1 Representación de la salida . . . 10
3.9.2 Familias de métricas . . . 10
3.9.3 Promediado y convenciones de reporte . . . 11
4 MFT-FiLM 13
4.1 Introducción . . . 13
4.2 Arquitectura y fundamentos del modelo . . . 14
4.2.1 Procesamiento de la entrada . . . 14
4.2.2 Modulación lineal por características . . . 14
4.2.3 Módulo FT-FilM . . . 15
4.2.4 Módulo IFT-FiLM . . . 15
4.2.5 Sistema de transcripción de notas . . . 16
4.2.6 Capas de salida y decodificación . . . 16
4.2.7 Función de pérdida . . . 17
4.2.8 Eficiencia computacional . . . 17
4.3 Configuración del experimento . . . 17
4.3.1 Entorno de computo . . . 17
4.3.2 Entorno de software . . . 18

4.3.3 Configuración de hiperparámetros 20
4.3.4 Preprocesamiento y caché 22
4.3.5 Entrenamiento 22
5 Transkun **25**
5.1 ?? ???? ???? ? ?? ?? 25
6 Conclusiones **27**
Bibliografía **29**

Apéndices
A Configuració del sistema **31**
A.1 Fase d'inicialització 31
A.2 Identificació de dispositius 31
B ??? ???????????? ???? **33**

Índice de figuras

Índice de tablas

3.1 Top 20 compositores en MAESTRO V3.0.0 7
3.2 Estadísticas de MAESTRO V3.0.0 por partición 8
3.3 Evolución de MAESTRO entre las versiones V1, V2 y V3 8
3.4 Condiciones de aceptación de un evento predicho como verdadero positivo según la variante de métrica de nota empleada. 11

4.1 Configuración de las capas de salida de cada rama del sistema MFT-FiLM. 16
4.2 Parámetros de configuración establecidos en `constant_macro.py` para la replicación del entrenamiento sobre MAESTRO v3.0.0. 20

CAPÍTULO 1
Introducción

????? ????????????? ????????????? ????????????? ????????????? ?????????????

1.1 Motivación

????? ????????????? ????????????? ????????????? ????????????? ?????????????

1.2 Objetivos

????? ????????????? ????????????? ????????????? ????????????? ?????????????

1.3 Estructura de la memoria

????? ????????????? ????????????? ????????????? ????????????? ?????????????

CAPÍTULO 2
Preliminares

????? ????????????? ????????????? ????????????? ????????????? ?????????????

2.1 ?? ???? ???? ? ?? ??

????? ????????????? ????????????? ????????????? ????????????? ?????????????

CAPÍTULO 3
MAESTRO

3.1 Introducción y contexto

Uno de los principales factores que limita el rendimiento de los sistemas de AMT basados en el aprendizaje profundo es la disponibilidad de datos de entrenamiento etiquetados con alta precisión y en cantidad suficiente para que los modelos sean capaces de generalizar. Históricamente, los datasets disponibles para esta tarea, como **MAPS** (*MIDI Aligned Piano Sounds*), no superaban las 70 horas de audio, lo que suponía una barrera significativa para el entrenamiento de arquitecturas modernas.

En este contexto, **Google Magenta** publicó en 2018 el dataset **MAESTRO** [1] (*Midi and Audio Edited for Synchronous Tracks and Organization*), que supuso un cambio de paradigma en el campo al ofrecer casi 200 horas de grabaciones de piano de alta calidad con alineación precisa entre audio y etiquetas MIDI. La publicación de MAESTRO demostró inmediatamente su impacto: el modelo **Onsets and Frames**, entrenado sobre MAESTRO alcanzó un **F1 de 67,43 %** en la evaluación sobre grabaciones Disklavier del conjunto MAPS, superando notablemente el **50,22 %** obtenido por el mismo modelo entrenado sobre MAPS únicamente. Desde entonces, MAESTRO se ha consolidado como el estándar de facto para la evaluación y comparación de sistemas de transcripción automática de piano.

En este trabajo se utiliza la versión **V3.0.0** [2] del dataset, publicada en 2020, que constituye una versión más depurada y es actualmente la referencia en la comunidad investigadora.

3.2 Origen: International Piano-e-Competition

Los datos que constituyen MAESTRO provienen de grabaciones recogidas durante diez ediciones de la International Piano-e-Competition, una competición de piano de nivel internacional celebrada anualmente en la que pianistas virtuosos interpretan sobre pianos de cola **Yamaha Disklavier** equipados con un sistema integrado de captura MIDI de alta precisión. Esta competición es la fuente exclusiva de todas las grabaciones del dataset, lo que dota al conjunto de datos de una coherencia excepcional en términos de calidad de la interpretación, condiciones acústicas y equipamiento.

3.2.1. Yamaha Disklavier

El elemento técnico que garantiza la calidad de MAESTRO es el piano Yamaha Disklavier. Es un piano de cola acústico convencional al que se le ha añadido un sistema

electrónico de captura que registra, con alta precisión temporal, el movimiento de cada tecla y pedal durante la interpretación. A diferencia de los sintetizadores o controladores MIDI, el Disklavier produce sonido acústico real al mismo tiempo que genera los datos MIDI, lo que garantiza que el audio captado por los micrófonos y el MIDI registrado sean inherentemente sincrónicos.

La alineación resultante entre la señal de audio y las etiquetas MIDI en MAESTRO es de aproximadamente ± 3 ms, muy inferior al umbral de percepción temporal humano (estimado en 20 ms), lo que hace que las etiquetas se consideren alineadas a nivel de fonación. Esta precisión es fundamental para entrenar modelos de transcripción: cualquier desalineación sistemática entre el onset de una nota en el audio y su etiqueta MIDI correspondiente introduce ruido en los gradientes durante el entrenamiento y degrada el rendimiento de los modelos.

3.3 Adquisición y anotación

3.3.1. Captura del audio

El audio se graba en calidad CD o superior, con una frecuencia de muestreo de 44,1 a 48 kHz y una profundidad de bit de 16 bits PCM[1] estéreo. Esta calidad es suficiente para representar con fidelidad toda la gama de frecuencias audibles relevantes para el piano (aproximadamente 28 Hz a 4 kHz para los fundamentales, con armónicos que se extienden hasta 16-20 kHz). Las grabaciones se almacenan sin compresión con pérdida, lo que preserva toda la información espectral original.

3.3.2. MIDI y anotación

El sistema MIDI del Disklavier captura automáticamente los siguientes eventos para cada interpretación:

- Velocidades de pulsación de teclas (*key strike velocities*, canal MIDI, nota, velocidad 0-127): codifican la dinámica de cada nota individual.
- Posición del pedal *sustain* (Control Change CC64): el pedal más utilizado en el piano, que prolonga el sonido de una tecla.
- Posición del pedal *sostenuto* (Control Change CC66): permite mantener seleccionadas únicamente las notas que están sonando en el momento de su activación.
- Posición del pedal *una corda* (Control Change CC67): pedal izquierdo, que reduce el volumen y modifica el timbre.

La inclusión de los tres pedales hace a MAESTRO uno de los pocos datasets de transcripción de piano con **anotación de pedal completa**, lo que resulta especialmente relevante para sistemas que aspiran a una transcripción expresiva o a la síntesis de audio realista a partir de MIDI.

Cada grabación se segmenta automáticamente en piezas musicales individuales y se anota con los siguientes metadatos:

- Compositor canónico (nombre normalizado).

[1] PCM (*Pulse Code Modulation*) es una técnica de representación digital de señales analógicas basada en el muestreo y la cuantización, ampliamente utilizada en sistemas de audio sin compresión.

- Título de la obra.
- Año de la grabación.
- Partición sugerida (train/validation/test).

3.4 Características del dataset

3.4.1. Composición

MAESTRO V3.0.0 está formado por un total de 1 276 grabaciones de interpretaciones de piano, que suman 198,7 horas de audio y más de 7,4 millones de eventos MIDI. El repertorio incluye obras de 60 compositores distintos que se asocian principalmente a la música clásica occidental entre los siglos XVII y XX.

Compositor	Grabaciones	Obras únicas	Duración (h)
Frédéric Chopin	201	107	26,22
Franz Schubert	186	96	35,82
Ludwig van Beethoven	146	95	26,92
Johann Sebastian Bach	145	93	13,77
Franz Liszt	131	91	20,27
Sergei Rachmaninoff	59	50	6,66
Robert Schumann	49	31	15,72
Claude Debussy	45	41	5,97
Joseph Haydn	40	29	4,72
Wolfgang Amadeus Mozart	38	35	5,60
Alexander Scriabin	35	26	5,35
Domenico Scarlatti	31	29	1,71
Felix Mendelssohn	28	8	4,85
Johannes Brahms	26	23	8,06
Franz Schubert / Franz Liszt	10	7	0,69
Isaac Albéniz	7	6	0,87
Modest Mussorgsky	6	2	3,17
Pyotr Ilyich Tchaikovsky / Mikhail Pletnev	6	6	0,80
Pyotr Ilyich Tchaikovsky	6	6	0,71
Muzio Clementi	6	6	0,87

Tabla 3.1: Los 20 compositores con mayor número de grabaciones en MAESTRO V3.0.0 [2].

La tabla 3.1 muestra la distribución típica de competiciones internacionales de piano, que tienden a centrarse en el período Romántico y en los grandes compositores del clasicismo tardío.

3.4.2. Particiones

Una característica fundamental de MAESTRO es su política de particionado, que distingue el dataset de otras colecciones de audio-MIDI. El criterio establecido por los autores es el siguiente: una misma composición no puede aparecer simultáneamente en más de una partición, aunque haya sido interpretada por distintos pianistas en periodos diferentes.

Esta decisión de diseño es crucial para garantizar la validez de los experimentos. Si múltiples interpretaciones de una misma obra aparecieran en entrenamiento y test, un modelo podría aprender características tímbricas o estructurales específicas de esa obra y

obtener resultados artificialmente buenos en la evaluación sin haber generalizado correctamente. De esta forma, los modelos se evalúan sobre obras no vistas en entrenamiento, lo que constituye una medida mucho más exigente y realista de rendimiento.

Las particiones están predefinidas y publicadas con el dataset, lo que garantiza la reproductibilidad de los experimentos y permite la comparación directa de distintos modelos. Esto es fundamental en un campo como AMT, en el que pequeñas diferencias en la partición de datos pueden producir variaciones significativas en los resultados reportados. En la tabla 3.2 se detallan las estadísticas principales por partición.

Partición	Grabaciones	Duración (h)	Tamaño (GB)	Notas (M)
Train	962	159,2	96,3	5,66
Validation	137	19,4	11,8	0,64
Test	177	20,0	12,1	0,74
Total	**1,276**	**198,7**	**120,2**	**7,40**

Tabla 3.2: Estadísticas de MAESTRO V3.0.0 desglosadas por partición.

3.5 Evolución del dataset

MAESTRO ya va por su tercera versión desde su publicación original en 2018. En la tabla 3.3 podemos ver características y principales cambios de cada versión.

Versión	Año	Grabaciones	Horas	Notas (M)	Principal novedad
V1.0.0	2018	1,184	172,3	6,18	Publicación original; datos 2004–2016.
V2.0.0	2019	1,282	201,2	7,13	Añade edición 2018; incorpora pedales *sostenuto* y *una corda*.
V3.0.0	2020	1,276	198,7	7,04	Elimina 6 grabaciones con acompañamiento orquestal.

Tabla 3.3: Comparativa de las tres versiones publicadas de MAESTRO.

La transición de V2.0.0 a V3.0.0 merece especial atención. Contrariamente a lo que podría esperarse, la versión más reciente no amplía el dataset, sino que realiza una corrección de calidad eliminando 6 grabaciones que no eran de piano solo. La presencia de estas grabaciones en versiones anteriores constituía una inconsistencia que podía afectar negativamente al entrenamiento de modelos especializados en piano solo. Esta corrección hace que las particiones de V3.0.0 no sean compatibles con las de V2.0.0, por lo que todos los experimentos de este trabajo se realizan íntegramente sobre V3.0.0 para garantizar la comparabilidad con los resultados publicados más recientes.

3.6 Distribución y licencia

MAESTRO se distribuye a través de Google Magenta en tres modalidades de descarga:

- **Audio + MIDI** (120,2 GB sin comprimir): contiene los archivos WAV y MIDI completos, junto con los metadatos en formato CSV y JSON.
- **Solo MIDI** ($\sim$ 57 MB): para tareas que no necesiten el audio.

- **Comprimido con pérdida** (MP3/FLAC): versión reducida para almacenamiento limitado.

El dataset se distribuye bajo licencia *Creative Commons Attribution Non-Commercial Share-Alike 4.0 International* (CC BY-NC-SA 4.0), que implica:

- Atribución obligatoria: cualquier uso debe citar la fuente original.
- Prohibido su uso con fines comerciales.
- Las obras derivadas deben distribuirse bajo la misma licencia.

Esta licencia es compatible con el uso académico en el contexto de un Trabajo de Fin de Grado.

3.7 Limitaciones

A pesar de sus fortalezas, MAESTRO presenta una serie de limitaciones que debemos tener en cuenta al interpretar los resultados de los sistemas entrenados sobre él.

Sesgo de repertorio. El repertorio es casi exclusivamente música clásica occidental de los siglos XVII-XX. Los modelos entrenados con MAESTRO no generalizan bien a géneros como el jazz, el pop o la música contemporánea con técnicas extendidas. Esto se debe a que las distribuciones de patrones armónicos, rítmicos y de velocidad son muy diferentes a las de la música clásica.

Sesgo de instrumento. Todas las grabaciones se realizaron sobre pianos Yamaha Disklavier del mismo modelo y en condiciones acústicas muy similares. Es por esto que los modelos entrenados con MAESTRO muestran una degradación significativa de rendimiento cuando se evalúan sobre grabaciones de pianos distintos en entornos con reverberación variable. Esto es especialmente relevante para aplicaciones del mundo real, donde el instrumento y la acústica son variables.

Ausencia de anotación de partitura. MAESTRO proporciona MIDI, pero no partitura en formato estándar como MusicXML o MEI. Para tareas de transcripción audio-a-partitura (*Audio-to-Score*), como parte del flujo de las tareas de AMT, es necesario complementar MAESTRO con datasets que incluyan partituras anotadas, como ASAP (*Aligned Scores and Performances*).

3.8 Relevancia para este trabajo

MAESTRO V3.0.0 es el dataset principal empleado para el entrenamiento y evaluación de los modelos de transcripción automática de piano desarrollados en este trabajo.

Los modelos de referencia del estado del arte, incluyendo los modelos empleados en este trabajo, reportan sus resultados sobre las particiones fijas de MAESTRO, lo que hace directamente comparables los experimentos aquí presentados con los resultados publicados en la literatura.

Su volumen de casi 200 horas con más de **7 millones de eventos MIDI** proporciona datos suficientes para el entrenamiento de arquitecturas basadas en *transformers* sin riesgo de sobreajuste, siempre y cuando se apliquen las técnicas de regularización adecuadas. Todos estos eventos tienen una precisión de alineación de ± 3 ms, garantizando la calidad de las etiquetas *onset*, *offset* y velocidad, que son las variables objetivo de los modelos de transcripción empleados.

Finalmente, las particiones fijas y predefinidas garantizan la reproductibilidad de los experimentos.

3.9 Evaluación estándar

La evaluación cuantitativa de los sistemas de transcripción automática de piano entrenados sobre MAESTRO sigue un protocolo consolidado en la comunidad, implementado mediante la librería `mir_eval`. Este protocolo es el empleado de forma invariante por todos los modelos del estado del arte que reportan resultados sobre este *dataset*, incluyendo los modelos reproducidos en este trabajo, MFT-FiLM y Transkun. Es por esto que en esta sección veremos cómo se evalúan ambos modelos.

3.9.1. Representación de la salida

Antes de aplicar las métricas, la salida de cada modelo, ya sea en forma de *piano roll* (MFT-FiLM) o en forma de intervalos decodificados (Transkun) se convierte en un conjunto de eventos MIDI discretos de la forma $\{(\text{tono}, t_{\text{onset}}, t_{\text{offset}}, v)\}$, donde el tono es un valor entero en $[21, 108]$ (las 88 teclas del piano estándar), los tiempos se expresan en segundos y la velocidad $v \in [0, 127]$ es un entero MIDI. La evaluación opera directamente sobre estos eventos discretos extraídos del MIDI de salida, comparándolos con los eventos del MIDI de *ground truth*, sin acceder en ningún momento a la representación interna del modelo, de forma que se garantiza la independencia de las métricas respecto a la arquitectura empleada.

3.9.2. Familias de métricas

Las métricas se organizan en dos categorías en función del nivel de abstracción al que operan.

Métricas a nivel de trama (*frame-level*)

Las métricas de trama cuantifican la capacidad del modelo para estimar qué teclas están activas en cada instante temporal, con independencia de cómo se delimiten los eventos. Para ello, tanto la predicción como el *ground truth* se convierten en matrices binarias de dimensiones $T \times 88$, con resolución temporal infinitesimal (equivalente a un piano roll continuo), comparando el estado de activación de cada tecla en cada instante.

Se calculan la **precisión** (P), la **exhaustividad** (R, *Recall*) y la **puntuación** F_1:

$$P = \frac{VP}{VP + FP} \qquad R = \frac{VP}{VP + FN} \qquad F_1 = \frac{2 \cdot P \cdot R}{P + R} \tag{3.1}$$

donde VP son los verdaderos positivos (instantes y teclas correctamente activadas), FP los falsos positivos (activaciones predichas incorrectamente) y FN los falsos negativos (activaciones del *ground truth* no detectadas). Estas métricas de trama son las más directamente relacionadas con la audición subjetiva de la transcripción, ya que capturan el porcentaje de tiempo en que la armonía reconstruida es correcta.

Métricas a nivel de nota

Las métricas de nota operan sobre los eventos discretos y evalúan la capacidad del modelo para segmentar correctamente los límites temporales de las notas. Hay tres variantes de complejidad creciente:

- **Nota con *onset*** (*Note Onset*): una nota predicha se considera un verdadero positivo si y solo si su tono es correcto y su instante de inicio está dentro de una tolerancia de ± 50 ms respecto al *onset* de *ground truth* correspondiente.
- **Nota con *onset* y *offset*** (*Note w/ Offset*): además de las condiciones anteriores, el instante de finalización de la nota predicha debe encontrarse dentro de una tolerancia igual al máximo entre 50 ms y el 20 % de la duración de la nota de referencia. Esta condición menos restrictiva para el *offset* refleja la mayor dificultad de detección del final de una nota, especialmente en presencia de pedal de *sustain*, cuyo efecto prolonga la resonancia del instrumento más allá del instante de liberación de la tecla.
- **Nota con *onset*, *offset* y velocidad** (*Note w/ Offset & Velocity*): añade una condición sobre la estimación de la dinámica. La velocidad predicha debe satisfacer $|v_{\text{pred}} - v_{\text{ref}}| \leq 0{,}1$ tras escalar ambos valores al intervalo $[0, 1]$, lo que equivale a una tolerancia absoluta de aproximadamente 12,7 unidades MIDI. Esta es la métrica más exigente y la más representativa de la calidad de una transcripción completa de *performance* MIDI.

La Tabla 3.4 resume las condiciones de aceptación de cada variante.

Tabla 3.4: Condiciones de aceptación de un evento predicho como verdadero positivo según la variante de métrica de nota empleada.

Métrica	Tono correcto	Tolerancia *onset*	Condición adicional
Frame	✓	—	—
Note Onset	✓	± 50 ms	—
Note w/ Offset	✓	± 50 ms	Tolerancia *offset*†
Note w/ Offset & Vel.	✓	± 50 ms	Tolerancia *offset*† + $\lvert\Delta v\rvert \leq 0{,}1$

† máx(50 ms, 20 % de la duración de la nota)

3.9.3. Promediado y convenciones de reporte

Todos los valores de las métricas se **promedian sobre el numero de piezas** del conjunto de evaluación, en lugar de calcularse de forma global sobre todas las notas. Este criterio es adoptado de forma uniforme por todos los trabajos de referencia [5, 9, 3].

Los valores de F_1 son los indicadores principales de rendimiento en la literatura de AMT, siendo la ***Note Onset*** F_1 la métrica más citada para comparar modelos, y la ***Note w/ Offset & Velocity*** F_1 la más representativa de la calidad de la transcripción completa.

CAPÍTULO 4
MFT-FiLM

4.1 Introducción

La transcripción de música de piano polifónico mediante redes neuronales se aborda contemporáneamente como un problema de detección multitarea (*Multitask Learning*, o MTL). En lugar de predecir la representación MIDI en un único paso, el sistema descompone la transcripción en varios objetivos interrelacionados: estimación del tono a nivel de frame (*frame-wise pitch*), la detección del inicio de la nota (*onset*), la detección del final (*offset*), y la estimación de la intensidad o velocidad (*velocity*). Dada la fuerte correlación entre estos eventos, el rendimiento general del sistema mejora significativamente al intercambiar información entre las diferentes tareas.

Sin embargo, las arquitecturas multitarea clásicas presentan limitaciones estructurales. Estrategias como la compartición estricta de parámetros (*hard parameter sharing*) pueden sufrir transferencias negativas debido a conflictos o discrepancias entre los resultados de distintas tareas. Por otro lado, la concatenación o adición directa de las características carece de los pesos adecuados para fusionar dinámicamente las representaciones extraídas por cada rama de la red.

Para abordar estos problemas, en este proyecto, la primera alternativa que se ha implementado es el modelo **MFT-FiLM** (*Multitask Frequency-Time Feature-Wise Linear Modulation*), propuesto por Wang *et al.* [9]. Este sistema introduce un sistema de acondicionamiento transversal basado en capas de modulación lineal (*Feature-wise Linear Modulation*, FiLM), diseñadas originalmente para tareas de razonamiento visual, y adaptadas aquí para la fusión de dependencias espectro-temporales en la señal del audio de entrada.

La arquitectura central del MFT-FiLM se basa en dos innovaciones fundamentales:

1. **Módulo FT-FiLM:** un bloque que se encarga de extraer información espectral y temporal de forma jerárquica, usando convoluciones unidimensionales a lo largo del eje de frecuencia y redes neuronales recurrentes (FG-LSTM) para capturar la dependencia temporal de los *frames* de audio. A diferencia de otras arquitecturas, este módulo aplica una transformación afín condicionada para auto-modular las características.

2. **Módulo IFT-FiLM (Inter-Task FiLM):** permite la interacción de información cruzada entre las distintas tareas. Mediante el uso de operaciones de transformación a nivel de tiempo y frecuencia, el sistema modula dinámicamente las características de una rama (por ejemplo *Frame offset*) utilizando como información condicional o auxiliar los parámetros generados por otras ramas (por ejemplo *Frame onset*)

La implementación de este modelo resulta especialmente atractiva para la extracción de *Performance* MIDI debido a su alta eficiencia computacional. A pesar de lograr métricas SOTA, el modelo MFT-FiLM requiere un volumen de parámetros muy pequeño (aproximadamente 1,2 millones), reduciendo la complejidad computacional y costes de entrenamiento sin sacrificar precisión acústica.

4.2 Arquitectura y fundamentos del modelo

4.2.1. Procesamiento de la entrada

Antes de que la señal de audio entre en la red neuronal, se aplica una cadena de preprocesamiento. El audio se convierte a mono y se remuestrea a 16 kHz. El oído humano no percibe las frecuencias de forma lineal: la diferencia perceptual entre 100 Hz y 200 Hz es la misma que entre 1000 Hz y 2000 Hz, ya que en ambos casos la segunda es el doble de la primera (diferencia de una octava musical). Las transformaciones tiempo-frecuencia clásicas como la **Transformada de Fourier de tiempo corto** (STFT) distribuyen las bandas de forma **lineal**. Esto implica que las frecuencias graves quedan con poca resolución y las frecuencias agudas con resolución excesiva, lo que no se corresponde con la estructura de la música ni con la percepción auditiva.

Es por esto que se usa la **Transformada Q Constante** (*Constant-Q Transform*, CQT), que distribuye las bandas de frecuencia en escala logarítmica, alineándose con la percepción musical y con la disposición de las teclas en el piano. La CQT presenta los siguientes parámetros de configuración:

- Frecuencia mínima de 27,5 Hz correspondiente a la tecla A0 (la tecla más grave de un piano estándar).
- 48 *bins* por octava, con un total de 352.
- Ventana de Hanning y salto de trama (*hop length*) de 20 ms.

El espectrograma CQT de dimensiones $(T \times 352)$, donde T es el número de *frames*, se remodela a (C_0, T, F) con $C_0 = 4$ y $F = 88$. De esta forma, los 352 *bins* se distribuyen en cuatro grupos de 88 frecuencias, en correspondencia biunívoca con las 88 teclas del piano [9]. Esta representación constituye la entrada de la capa lineal inicial del modelo.

4.2.2. Modulación lineal por características

El mecanismo *Feature-Wise Linear Modulation* (FiLM) fue propuesto inicialmente por Perez *et al.* (2018) para tareas de razonamiento visual. La idea central consiste en condicionar las activaciones de una red neuronal mediante una transformación afín por cada elemento, usando información de contexto generada por un módulo externo denominado *FiLM Generator*. Definimos formalmente la operación como:

$$\mathrm{FiLM}(x,\, \gamma,\, \beta) = \gamma \odot x + \beta \tag{4.1}$$

donde x es el mapa de características de entrada, γ y β son los parámetros de escala y desplazamiento generados dinámicamente, y $\odot$ es el producto de Hadamard (multiplicación elemento a elemento). El principal beneficio de este mecanismo respecto a la suma o concatenación directa de características es que γ y β se adaptan en función del contexto en cada paso de inferencia, permitiendo al modelo ponderar y suprimir canales de forma dinámica en lugar de aplicar pesos fijos.

4.2.3. Módulo FT-FilM

El módulo **FT-FiLM** (*Frequency-Time FiLM*) adapta el mecanismo descrito en la ecuación (4.1) para extraer dependencias espectrales y temporales de forma jerárquica a partir del espectrograma CQT. Su operación se define como:

$$\text{FT-FiLM}(x) = f_{FT}(x) \odot x + h_{FT}(x) \tag{4.2}$$

donde f_{FT} y h_{FT} son generadores de los parámetros de modulación γ y β respectivamente, extrayendo la información espectro-temporal en tres etapas secuenciales:

Convolución en el eje de frecuencia. El mapa de características de entrada, de dimensiones (C, T, F), se convoluciona con un kernel 1D de tamaño $2F - 1 = 175$ sobre el eje de frecuencia. Este kernel se diseña con una longitud igual al eje completo para capturar simultáneamente las relaciones armónicas entre todos los *bins* de frecuencia del piano, algo que las convoluciones de kernel reducido no pueden lograr [9].

Transformación de canales. Los canales se expanden de C a $6C$ mediante una convolución 1D con kernel de tamaño 1, y el resultado se divide en dos tensores z y z', cada uno de C canales, que servirán respectivamente como parámetros de escala y desplazamiento de la modulación afín.

Modelado temporal con FG-LSTM. Los tensores z y z' se procesan con dos redes recurrentes **FG-LSTM** (*Frequency-Grouped LSTM*) bidireccionales independientes, cada una con 32 unidades ocultas, que capturan las dependencias temporales entre *frames* consecutivos. El uso de LSTM bidireccional incorpora tanto el contexto pasado como el futuro en cada instante de la secuencia.

Tras estas tres etapas, el mapa de entrada original x se modula aplicando la transformación afín de la ecuación (4.2), con una capa de *dropout* de tasa 0,1 antes de la modulación final. El resultado es una representación auto-condicionada que integra información espectral y temporal de forma jerárquica y separada, lo que permite mantener un número de parámetros reducido.

4.2.4. Módulo IFT-FiLM

El módulo IFT-FiLM (*Inter-Task FT-FiLM*) extiende el FT-FiLM para permitir la interacción entre ramas de distintas tareas. Lo formulamos como:

$$\text{IFT-FiLM}(x,\, x') = f_{IFT}(x,\, x') \odot x + h_{IFT}(x,\, x') \tag{4.3}$$

donde x es el mapa de características de la rama principal y x' es la información auxiliar proveniente de otra rama. La diferencia estructural respecto al FT-FiLM reside en el paso de transformación de canales: los tensores z y z' generados por la rama auxiliar se concatenan a los de la rama principal a lo largo del eje de canales antes de reducirlos de vuelta a C mediante una convolución 1D con kernel de tamaño 1.

Un detalle de implementación relevante es que los tensores auxiliares se **desconectan del grafo de cómputo** (`detach`) antes de ser incorporados. Esta operación impide que el gradiente fluya de vuelta hacia la rama auxiliar a través de esta conexión, de forma que evitamos inestabilidades en la optimización cuando dos ramas se condicionan mutuamente. El módulo IFT-FiLM puede interpretarse como una red neuronal dinámica: los

parámetros de modulación no son fijos, sino que se generan en cada paso en función de las entradas de ambas ramas, permitiendo que la red adapte su comportamiento de acuerdo con el contexto conjunto de las tareas involucradas.

4.2.5. Sistema de transcripción de notas

El sistema completo de transcripción está compuesto por **tres ramas paralelas**: *onset*, *velocity* y *frame+offset* [9]. Cada rama apila cuatro bloques FT-FiLM o IFT-FiLM en cascada, y el flujo de información cruzada entre ramas sigue la siguiente lógica:

- La **rama *onset*** emplea bloques FT-FiLM puros (auto-condicionamiento), ya que detectar el comienzo de una nota es la subtarea más discriminativa del sistema.
- En cada uno de los cuatro niveles, los tensores z y z' generados por la rama *onset* se desconectan y se reenvían a las ramas *velocity* y *frame*, que los incorporan mediante bloques IFT-FiLM.
- De esta forma, el diseño produce **cuatro niveles de interacción cruzada**, desde representaciones de bajo nivel espectral hasta semánticas de alto nivel, enriqueciendo progresivamente las representaciones de cada subtarea.

El flujo unidireccional de información (onset $\rightarrow$ velocity, onset $\rightarrow$ frame) está justificado por el hecho de que la señal de *frame* combina información de notas sostenidas y de finales, lo que podría interferir de forma negativa en la detección precisa de inicios si se utilizara como entrada de la rama *onset*. Las dimensiones de los canales son $C = 64$ en las ramas *onset* y *velocity*, y $C = 32$ en la rama *frame+offset*, lo que fomenta la minimización de los parámetros del modelo.

4.2.6. Capas de salida y decodificación

Cada rama dispone de una capa de salida que ajusta el número de canales al objetivo de la subtarea correspondiente. La capa sigue el esquema: expansión a $2C$ canales, seguida de una capa de *dropout* con tasa 0,1, activación GELU (que actúa como una versión suavizada de una ReLU), y compresión al número de canales objetivo [9]. La tabla 4.1 resume las configuraciones de salida de cada rama.

Tabla 4.1: Configuración de las capas de salida de cada rama del sistema MFT-FiLM.

Rama	Canales de salida	Activación	Dimensión del piano roll
Onset	1	Sigmoide	$T \times 88$
Frame+Offset	2	Sigmoide	$T \times 88\,(\times 2)$
Velocity	128	—	$T \times 88 \times 128$

Durante la inferencia se aplica una estrategia de **medio paso** (*half stride*), que procesa el audio en ventanas con un solapamiento del 50 % para mitigar los artefactos en los bordes de cada bloque. El algoritmo de decodificación heurístico convierte los piano rolls de probabilidad en eventos de nota discretos (tono, inicio, final, velocidad), con una tolerancia de 50 *ms* en los inicios; y del máximo entre 50 *ms* y el 20 % de la duración en los finales.

4.2.7. Función de pérdida

La función de pérdida del sistema es la suma de las pérdidas individuales de cada subtarea:

$$\ell = \ell_{\text{onset}} + \ell_{\text{frame}} + \ell_{\text{offset}} + \ell_{\text{velocity}} \tag{4.4}$$

Las pérdidas de *onset*, *frame* y *offset* se calculan mediante **entropía cruzada binaria** (BCE), apropiada para clasificación binaria píxel a píxel del piano roll:

$$\ell_{\text{onset}} = \sum_{k=1}^{K} \sum_{t=1}^{T} \ell_{\text{BCE}}(I_{\text{onset}}(t,k),\ P_{\text{onset}}(t,k)) \tag{4.5}$$

y análogamente para ℓ_{frame} y ℓ_{offset}. La pérdida de velocidad se calcula mediante **entropía cruzada categórica** (CCE) sobre 128 clases, dado que la velocidad MIDI es un valor entero en el intervalo $[0, 127]$:

$$\ell_{\text{velocity}} = \sum_{k=1}^{K} \sum_{t=1}^{T} \ell_{\text{CCE}}\big(I_{\text{velocity}}(t,k),\ P_{\text{velocity}}(t,k)\big) \tag{4.6}$$

donde I y P representan los valores de *ground truth* y predicción respectivamente, $K = 88$ es el número de teclas del piano y T el número de *frames* [9]. La ausencia de factores de ponderación entre las cuatro pérdidas es una elección de diseño deliberada por los autores, cuya validez queda respaldada por el estudio de ablación presentado en la sección de resultados [9].

4.2.8. Eficiencia computacional

Una de las aportaciones más destacadas del modelo es su reducido tamaño. Con tan solo 1,2 **millones de parámetros**, el MFT-FiLM se sitúa como uno de los modelos de transcripción de piano más eficientes del estado del arte. En términos de operaciones de multiplicación-Acumulación (MACs), calculadas para una salida de 128 *frames* con la biblioteca `ptflops`, el MFT-FiLM registra una complejidad notablemente inferior a la de otros modelos como el hFT-Transformer o el SemiCRF V2 [9].

Esta eficiencia no sacrifica capacidad expresiva gracias al diseño jerárquico del generador FiLM, que desacopla el procesamiento en los ejes de frecuencia, canal y tiempo en lugar de operarlos conjuntamente. La separación de estas dimensiones permite reutilizar módulos ligeros (convoluciones 1D y LSTMs de baja dimensionalidad) en lugar de recurrir a mecanismos de atención de complejidad cuadrática, sin renunciar a la capacidad de capturar dependencias espectrales y temporales de largo alcance.

4.3 Configuración del experimento

4.3.1. Entorno de computo

El entrenamiento del modelo MFT-FiLM se llevó a cabo en el clúster de cómputo de alto rendimiento **tensor**, que pertenece al Departamento de Sistemas Informáticos y Computación (DSIC) de la Universidad Politécnica de Valencia. Este sistema está formado por un único nodo equipado con 503 GB **de memoria RAM** y cuatro unidades de procesamiento gráfico **NVIDIA L40S** con 48 GB de memoria GDDR6 ECC cada una. Las GPUs L40S son tarjetas de la familia *Ada Lovelace*, orientadas específicamente a cargas

de trabajo de inteligencia artificial sostenidas en entornos de servidor. A diferencia de las tarjetas de consumo de la misma generación, estas incorporan memoria ECC (*Error-Correcting Code*), que detecta y corrige errores de bit durante el cómputo, algo que resulta realmente útil en trabajos de larga duración como el entrenamiento de redes neuronales.

El hardware empleado en el artículo original consistía en dos tarjetas NVIDIA GeForce RTX 4090 de 24 *GB* de VRAM cada una. La L40S cuenta individualmente con 48 *GB*, lo que permitió alojar el modelo con lotes de tamaño deseado sin problemas.

El acceso se realiza mediante SSH a través del nodo de entrada `tensor.dsic.upv.es`, autenticado con credenciales institucionales. El sistema de gestión de trabajos es **SLURM** (*Simple Linux Utility for Resource Management*), que organiza la ejecución de trabajos mediante dos colas principales: `docencia`, para trabajos de duración inferior a 24 horas; y `long`, para trabajos de hasta 72 horas. Dada una duración de entrenamiento estimada por los autores de 43 horas, fue necesario usar la cola `long`.

4.3.2. Entorno de software

Todas las dependencias se encapsularon en un entorno virtual de Python para garantizar la reproductibilidad del experimento y para aislar las versiones de las librerías del sistema global del clúster. El uso de entornos virtuales es una práctica estándar en proyectos de aprendizaje automático, ya que distintos proyectos pueden requerir versiones incompatibles de las mismas librerías. El proceso de creación fue el siguiente:

1. Creación del entorno virtual con el intérprete Python 3.12 disponible en el clúster:

```
mkdir -p ~/venv
python3 -m venv ~/venv/MFT-FiLM
source ~/venv/MFT-FiLM/bin/activate
```

2. Instalación de PyTorch con soporte para CUDA 12.6, versión de controladores disponible en tensor:

```
pip install torch torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/cu126
```

3. Instalación del resto de dependencias del repositorio:

```
pip install prettymidi mireval librosa==0.9.2 \
    numpy scipy joblib tensorboard
```

Después de preparar el entorno, se clonó el repositorio oficial público desde el GitHub oficial [9], y se creó un enlace simbólico para trabajar con MAESTRO previamente descargado. Durante la realización de pruebas para comprobar el funcionamiento del código original encontramos ciertas dificultades. En los siguientes apartados veremos en detalle los problemas técnicos más relevantes y las soluciones adoptadas para resolverlos.

Problema 1: Obsolescencia de dependencias

La reproductibilidad en aprendizaje automático aplicado a la recuperación de información musical es altamente sensible a las versiones de las librerías de procesamiento de señal. Para garantizar que las matrices de los espectrogramas CQT generados coincidieran con las reportadas en el artículo original, fue estrictamente necesario usar la versión

0.9.2 de la librería `librosa`. Las versiones más recientes (a partir de la 0.10) introdujeron modificaciones en el cómputo del *padding* interno del CQT, lo que produce una diferencia de uno o dos *frames* entre el tensor del audio y el tensor de etiquetas MIDI generado a partir del mismo archivo. Dado que el código asume una correspondencia exacta entre ambas dimensiones temporales, este desajuste provocaba un error fatal durante la inicialización del *DataLoader*.

El clúster de tensor opera con un intérprete **Python 3.12**. A partir de esta versión, las herramientas de construcción base del ecosistema Python (`setuptools` v70+) eliminaron por completo el módulo `pkg_resources`, considerado obsoleto. Sin embargo, la versión 0.9.2 de `librosa` posee una dependencia rígida en dicho módulo para localizar e instanciar sus archivos internos de configuración. Esto generó un bloqueo de entorno: el modelo requería la librería heredada para mantener la integridad dimensional de los datos, pero el intérprete moderno de Python impedía su carga, abortando el proceso de entrenamiento antes incluso de la fase de carga de datos.

Para solventarlo sin alterar la versión global de Python de tensor y sin comprometer la versión de `librosa`, se aplicó una degradación forzada de las herramientas de construcción del entorno virtual:

```
pip install "setuptools==69.5.1" --force-reinstall
```

La versión 69.5.1 es la última versión de transición que conserva el módulo necesario `pkg_resources` íntegro en el espacio de usuario. Esta acción creó un puente de compatibilidad que permitió ejecutar código de procesamiento de señal de generación anterior sobre un intérprete Python de última generación, sin necesidad de modificar ningún archivo del código fuente del repositorio ni alterar el entorno global del sistema.

Problema 2: Incompatibilidad de la API de PyTorch y corrupción de la caché

Como veremos posteriormente, el entrenamiento requiere una fase previa de **preprocesamiento masivo paralelo** para extraer todos los espectrogramas y los *piano rolls* de las etiquetas MIDI, almacenándolos en archivos binarios `.pth` en disco (caché). Tras finalizar este proceso, la clase constructora del dataset (`CacheDataset`) debía cargar dichos tensores en memoria para ejecutar el bucle de entrenamiento del modelo.

Al iniciarse la fase de entrenamiento, el código produjo una excepción crítica:

```
AssertionError: Size mismatch between tensors
```

En tareas de recuperación de información musical, este tipo de error suele indicar un desajuste en el número de *frames* entre el espectrograma de audio y la correspondiente matriz de etiquetas (por ejemplo, un audio que genera 1.000 *frames* y cuyo *piano roll* genera 1.001 debido a redondeos en aritmética de coma flotante). Sin embargo, tras analizar en profundidad el código fuente de los autores, se identificó que la causa era un **defecto de diseño arquitectónico** expuesto por la actualización de PyTorch.

Los autores definieron la clase `CacheDataset` heredando de `TensorDataset`. No obstante, en su método inicializador, invocaban a la clase padre sin pasar ningún tensor (`super(CacheDataset, self).__init__()`), ya que gestionaban la carga en memoria de forma personalizada. Las versiones antiguas de PyTorch (serie 1.x) aceptaban esta inicialización vacía de forma silenciosa. Las versiones modernas (serie 2.x, empleadas en tensor) incorporan validaciones internas que asumen que la ausencia de tensores en la inicialización implica un desajuste dimensional, lanzando la excepción de forma sistemática.

Adicionalmente, las interrupciones abruptas producidas durante los primeros intentos corrompieron parcialmente los archivos `.pth` de caché generados, dejando el sistema en un estado irrecuperable que exigió intervención manual.

Para solventar los problemas, por un lado se modificó la declaración de herencia de la clase `CacheDataset` en el archivo `data/dataload.py`, sustituyendo la herencia de `TensorDataset` por la clase base estándar `torch.utils.data.Dataset`:

```
# Antes:
class CacheDataset(TensorDataset):
    def __init__(self, ...):
        super(CacheDataset, self).__init__()
        ...

# Después:
class CacheDataset(torch.utils.data.Dataset):
    def __init__(self, ...):
        super(CacheDataset, self).__init__()
        ...
```

Al heredar de la clase base genérica, se evitó la validación estricta de tensores en la inicialización, delegando la responsabilidad de la gestión dimensional a los métodos `__getitem__` y `__len__`, que ya estaban correctamente implementados por los autores. Por otro lado se eliminó completamente el árbol de directorios de caché corrupta y se regeneraron los archivos desde cero.

4.3.3. Configuración de hiperparámetros

El archivo central de configuración del repositorio es `constant_macro.py`, que controla todos los hiperparámetros y rutas del sistema. Para replicar las condiciones descritas en el artículo original, se ajustaron los parámetros que se muestran en la Tabla 4.2.

Tabla 4.2: Parámetros de configuración establecidos en `constant_macro.py` para la replicación del entrenamiento sobre MAESTRO v3.0.0.

Parámetro	Valor	Descripción
`EPOCH_RANGE`	60	Número total de épocas de entrenamiento
`BATCH_SIZE`	10	Número de muestras por lote
`LEARNING_RATE`	0,0005	Tasa de aprendizaje del optimizador
`DATASET_NAME`	`maestro-v3.0.0`	Versión del *dataset*
`DATASET_PATH`	ruta absoluta	Directorio raíz de los datos
`MODEL_PATH`	ruta absoluta	Directorio de salida del modelo
`MODEL_NAME`	`MTFiLM`	Identificador del modelo
`SCHEDULE_IDX`	`2x`	Objetivo: transcripción de notas

A continuación veremos la significación y se justificará la elección de los parámetros más relevantes.

Optimizador y tasa de aprendizaje

El optimizador empleado es **Adam** (*Adaptive Moment Estimation*), considerado estándar en el entrenamiento de redes para tareas de procesado de señal. Este optimizador

mantiene para cada parámetro del modelo, estimaciones del primer momento (media exponencial de los gradientes pasados, m_t) y del segundo momento (media exponencial de los cuadrados de los gradientes, v_t), actualizando cada parámetro θ según:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{4.7}$$

donde $\hat{m}_t$ y $\hat{v}_t$ son las estimaciones corregidas por sesgo de ambos momentos, α es la tasa de aprendizaje global, y ϵ es un término de estabilidad numérica. La principal ventaja de Adam sobre el descenso de gradiente estocástico clásico (SGD) es que la tasa de aprendizaje efectiva de cada parámetro se adapta automáticamente en función de la magnitud histórica de sus gradientes. Los parámetros que reciben gradientes de alta magnitud con frecuentemente se actualizan menos veces, mientras que aquellos con gradientes escasos o de baja magnitud se actualizan más veces. Este comportamiento es especialmente beneficioso en arquitecturas multitarea como el MFT-FiLM, donde diferentes ramas de la red pueden tener gradientes de magnitud muy dispar.

La tasa de aprendizaje global se fijó en $\alpha = 5 \times 10^{-4}$, valor utilizado por los autores originales [9].

Tamaño de lote

El **tamaño de lote** (*batch size*) de 10 espectrogramas es el valor reportado en el artículo [9], y difiere del valor predeterminado del repositorio (4 muestras). El *batch size* es relevante desde dos puntos de vista: la estabilidad del gradiente y el hardware.

Desde el punto de vista de la optimización, valores elevados producen estimaciones del gradiente con menor varianza, lo que se traduce en actualizaciones de parámetros más estables y convergencia más suave. Lotes excesivamente grandes pueden conducir a mínimos agudos con peor capacidad para generalizar. El valor de 10 representa un equilibrio pragmático entre ambos extremos para este dominio.

Desde el punto de vista del hardware, cada muestra de entrenamiento es un tensor de espectrograma de dimensiones $(4, T, 88)$, donde $T = 400$ *frames* (correspondiente a una ventana de 8 segundos con *hop* de 20 ms). Un lote de 10 de estas muestras ocupa aproximadamente 40 GB de la VRAM de la GPU L40S, situando la utilización de memoria cerca del límite físico de 48 GB disponibles. Lotes de mayor tamaño habrían requerido el uso de más gráficas para alojar el modelo durante el entrenamiento.

Épocas y guardado

El número de **épocas** se fijó en 60, conforme a la configuración descrita en el artículo para el entrenamiento sobre MAESTRO [9]. Una época representa una pasada completa del optimizador sobre todos los ejemplos del conjunto de entrenamiento; con 7 186 lotes de 10 muestras, cada época implica 7 186 actualizaciones de los parámetros del modelo. El número de épocas determina la cantidad total de actualizaciones de gradiente y, por tanto, el grado de especialización del modelo sobre el conjunto de entrenamiento.

Para minimizar el riesgo de sobreajuste, se implementó una estrategia de **guardado dinámico por validación**. Al principio (10 épocas), la evaluación del modelo sobre el conjunto de validación se realiza cada 5 épocas. Una vez supera un umbral mínimo de convergencia, la validación se realiza cada época, guardando un *checkpoint* únicamente cuando se observa una mejora respecto al mejor valor histórico de pérdida en validación. De esta forma, el modelo final seleccionado no es necesariamente el de la última época completada, sino el que mejor se comportó en validación.

4.3.4. Preprocesamiento y caché

Antes del entrenamiento, se ejecuta una fase de **preprocesamiento y caché** del *dataset*. Durante esta fase se transforma cada audio del conjunto MAESTRO en su espectrograma CQT mediante la librería `librosa`, y los archivos MIDI correspondientes son procesados mediante `pretty_midi` para generar los *piano rolls* de *ground truth*.

Adicionalmente, la duración de cada nota en el *ground truth* se extiende en función de la información de los pedales de *sustain* y *sostenuto*, siguiendo el siguiente criterio: una nota se prolonga hasta que el valor del pedal de sustain en el archivo MIDI cae por debajo de 64, o hasta que la misma tecla es pulsada de nuevo. Este paso es crucial para la fidelidad del *ground truth*, ya que en interpretaciones de piano con pedal, las notas resuenan durante más tiempo del que dura la pulsación física de la tecla; ignorar esta información produciría un *ground truth* sistemáticamente más corto que el audio real.

El proceso se paraleliza mediante la librería `joblib`, y el resultado de la ejecución es un conjunto de archivos binarios que serán reutilizados directamente en todos los entrenamientos subsiguientes, eliminando el coste de reprocesamiento.

4.3.5. Entrenamiento

Dado que el tiempo estimado de entrenamiento excedía ampliamente el límite de 24 horas de la cola `docencia`, el trabajo se sometió a la cola `long` de SLURM. Se solicitó una única GPU L40S, ya que, como hemos visto, es suficiente para alojar el modelo con el *batch size* seleccionado.

Durante el entrenamiento se observó un patrón de utilización de la GPU con oscilaciones periódicas de uso entre el 0 % y el 100 %, en lugar de un valor cercano al máximo continuado. Este fenómeno, conocido como *GPU starvation* o cuello de botella de entrada/salida (*I/O bottleneck*), se produce cuando la velocidad de cómputo de la GPU es significativamente superior a la velocidad con que la CPU puede preparar y transferir nuevos lotes de datos a la VRAM.

El mecanismo es el siguiente: una vez que la GPU termina de procesar un lote de 10 espectrogramas —operación que, dada la compacidad del modelo (1,2 M de parámetros), se completa en pocos segundos—, el proceso queda bloqueado a la espera de que la CPU lea el siguiente lote desde los archivos `.pth` en disco y lo transfiera a la VRAM a través del bus PCIe. Durante ese intervalo de espera, la utilización de la GPU cae al 0 %. En la configuración empleada, este fenómeno fue particularmente pronunciado debido a que tensor solo tiene un nodo, compartido por todos los usuarios, y no se puede forzar el `DataLoader` para usar continuamente todos los `num_workers` disponibles.

Como consecuencia directa de este cuello de botella, la duración efectiva de cada época fue de aproximadamente **83 minutos**, frente a los 43,5 minutos reportados por los autores originales con dos GPUs RTX 4090. En el *setup* original, la paralelización entre dos GPUs permitía solapar parcialmente la fase de transferencia de datos con el cómputo sobre la segunda tarjeta, mitigando significativamente este efecto y casi duplicando el rendimiento efectivo.

El clúster tensor impone una duración máxima de **72 horas** para los trabajos sometidos a la cola `long`. Dado que cada época requirió aproximadamente 83 minutos —prácticamente el doble del tiempo estimado inicialmente—, el trabajo de entrenamiento fue interrumpido por SLURM al alcanzar dicho límite temporal, habiendo completado únicamente **30 de las 60 épocas planificadas**.

A pesar de no haber completado el ciclo de entrenamiento establecido en el artículo, el mecanismo de validación dinámica y guardado automático de *checkpoints* preservó el mejor modelo alcanzado durante las 30 épocas ejecutadas. Este modelo resultó lo suficientemente bueno en la evaluación, obteniendo resultados muy cercanos al estado del arte, que analizaremos en la siguiente sección.

CAPÍTULO 5
Transkun

????? ????????????? ????????????? ????????????? ????????????? ?????????????

5.1 ?? ???? ???? ? ?? ??

????? ????????????? ????????????? ????????????? ????????????? ?????????????

CAPÍTULO 6
Conclusiones

????? ????????????? ????????????? ????????????? ????????????? ?????????????

Bibliografía

[1] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman y Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO dataset. *International Conference on Learning Representations (ICLR)*, 2019. Disponible en https://arxiv.org/abs/1810.12247.

[2] Google Magenta. The MAESTRO dataset v3.0.0, 2020. Consultado en marzo de 2026 en https://magenta.tensorflow.org/datasets/maestro.

[3] Yujia Yan, Frank Cwitkowitz y Zhiyao Duan. Transkun: Transcription of keyboard music with attention-based U-Net. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, 2023. Disponible en https://arxiv.org/abs/2306.10801.

[4] Qiuqiang Kong, Bochen Li, Xuchen Song, Yuan Wan y Yuxuan Wang. High-resolution piano transcription with pedals by regressing onset and offset times. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3707–3717, 2021. Disponible en https://arxiv.org/abs/2010.01815.

[5] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore y Douglas Eck. Onsets and frames: Dual-objective piano transcription. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, págs. 50–57, 2018. Disponible en https://arxiv.org/abs/1710.11153.

[6] ByteDance Research. Piano transcription. Consultado en marzo de 2026 en https://github.com/bytedance/piano_transcription.

[7] Francesco Foscarin, Katharina Hoedt, Gerhard Widmer y Maarten Grachten. Towards musically informed evaluation of piano transcription models. *arXiv preprint*, 2024. Disponible en https://arxiv.org/abs/2406.08454.

[8] Qi Wang, Mingkuan Liu, Changchun Bao y Maoshen Jia. Harmonic-Aware Frequency and Time Attention for Automatic Piano Transcription. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, 2024.

[9] Qi Wang, Mingkuan Liu y Maoshen Jia. Multitask Frequency-Time Feature-Wise Linear Modulation for Piano Transcription With Pedal. *IEEE/ACM Transactions on Audio, Speech and Language Processing*, vol. 33, pp. 1049–1062, 2025.

APÉNDICE A
Configuració del sistema

????? ????????????? ????????????? ????????????? ????????????? ?????????????

A.1 Fase d'inicialització

????? ????????????? ????????????? ????????????? ????????????? ?????????????

A.2 Identificació de dispositius

????? ????????????? ????????????? ????????????? ????????????? ?????????????

APÉNDICE B
??? ????????????? ????

????? ????????????? ????????????? ????????????? ????????????? ?????????????